

PROYECTO DE SOFTWARE: WEB-SCRAPING

María Arribas Ballesteros

3^a Ingeniería Matemática

WEB SCRAPING



Contents

1	Introducción a la práctica	3
2	Parte 1: Uso de PLY para análisis léxico y sintáctico de HTML	3
2.1	Resumen del uso que ha realizado de PLY	4
2.2	Decisiones de diseño tomadas en la práctica y cómo se han implementado . . .	5
2.3	Descripción detallada de las pruebas definidas, resultados y justificación	6
2.4	Grafo del autómata del analizador léxico	9
3	Parte 2: Uso de BeautifulSoup para análisis de la estructura DOM	13
3.1	Resumen del uso que ha realizado de BeautifulSoup	13
3.2	Decisiones de diseño tomadas en la práctica y cómo se han implementado	13
3.3	Descripción detallada de las pruebas realizadas, objetivos, resultados y validación	14
4	Conclusiones	17
5	Completitud de los tests	18

1 Introducción a la práctica

El *Web-Scraping* es una técnica informática que permite extraer información de páginas web de forma automatizada. En la actualidad, esta práctica es fundamental para tareas como la recopilación de datos, comparación de precios, monitorización de contenidos o indexación por parte de motores de búsqueda.

Desde el punto de vista académico, el *Web-Scraping* se relaciona directamente con conceptos fundamentales de la Teoría de la Computación, como el análisis léxico y sintáctico, que son la base para procesar lenguajes formales. En esta práctica, se aplican estos conceptos teóricos al análisis del lenguaje HTML, utilizado para estructurar las páginas web.

El objetivo principal de esta práctica es implementar dos enfoques para analizar documentos HTML con el fin de extraer URLs de hiperenlaces e imágenes:

- **Enfoque 1:** Uso de analizadores léxicos y sintácticos basados en `PLY` (*Python Lex-Yacc*), que permiten procesar el texto de una página web a nivel de tokens y estructura gramatical. Esto implica diseñar un lexer para identificar elementos léxicos y un parser para validar la estructura del HTML, especialmente el balanceo de etiquetas.
- **Enfoque 2:** Uso de la librería `BeautifulSoup`, que facilita la construcción y navegación del árbol DOM (*Document Object Model*) de una página HTML. Este método simplifica la extracción de información mediante métodos de búsqueda y navegación de nodos, siendo capaz de manejar HTML mal formado o complejo.

Además de la extracción de enlaces e imágenes, la práctica incluye verificar que la página web analizada está correctamente estructurada en cuanto al balanceo de sus etiquetas, y obtener estadísticas de uso de ciertas etiquetas HTML mediante el análisis DOM.

El desarrollo de esta práctica contribuye a consolidar los conocimientos sobre autómatas, gramáticas, *parsing* y estructura de lenguajes formales, aplicándolos a un caso real con aplicaciones prácticas en ingeniería informática.

2 Parte 1: Uso de `PLY` para análisis léxico y sintáctico de HTML

Fundamentos teóricos sobre analizadores léxicos y sintácticos

Un **analizador léxico** es un componente fundamental en el procesamiento de lenguajes formales, cuya función principal es transformar una secuencia de caracteres de entrada (como un texto o código fuente) en una secuencia de unidades mínimas llamadas *tokens*. Cada token representa un conjunto de caracteres con significado semántico común, por ejemplo, palabras clave, identificadores, números, operadores o símbolos especiales.

Para identificar estos tokens, el analizador léxico utiliza reglas basadas en **expresiones regulares** o bien en autómatas finitos. Las expresiones regulares permiten describir patrones de caracteres, como una palabra que empieza con una letra seguida de dígitos, o una etiqueta HTML concreta. Los autómatas finitos, por su parte, son máquinas abstractas que recorren el texto carácter a carácter para determinar si una cadena cumple con un patrón definido.

Ejemplo: en un lenguaje de programación como Python, el analizador léxico puede reconocer que la cadena de caracteres `"if"` corresponde a un token de palabra clave `TOKEN_IF`, y que la cadena `"variable1"` es un token identificador `TOKEN_ID`.

Por otro lado, un **analizador sintáctico** recibe la secuencia de tokens generada por el analizador léxico y verifica si estos tokens están organizados siguiendo una gramática formal

específica. Esta gramática describe las reglas de construcción válidas del lenguaje, permitiendo construir estructuras jerárquicas llamadas *árboles sintácticos* o *árboles de derivación*. Estos árboles representan la organización lógica del texto y son esenciales para entender el significado y estructura del lenguaje.

En el contexto del análisis de HTML, el analizador sintáctico comprueba que las etiquetas HTML están correctamente anidadas y balanceadas, por ejemplo, que cada etiqueta `<div>` tiene una etiqueta de cierre correspondiente `</div>`, y que el orden de apertura y cierre respeta las reglas del lenguaje.

La herramienta **PLY** (*Python Lex-Yacc*) permite implementar ambos tipos de analizadores en Python. Primero, el *lexer* se encarga de dividir el texto en tokens según expresiones regulares, y luego el *parser* usa una gramática definida para validar la secuencia de tokens y construir la estructura sintáctica.

Este flujo —de *lexer* a *parser*— es fundamental para procesar lenguajes complejos, ya que separa la tarea de identificar unidades básicas (tokens) de la tarea de verificar la organización lógica y estructural.

Autómatas finitos y expresiones regulares

Un **autómata finito** es un modelo matemático utilizado para reconocer patrones dentro de cadenas de caracteres. Se compone de un conjunto finito de estados, un estado inicial, un conjunto de estados de aceptación y transiciones entre estados que se activan con la lectura de símbolos.

Existen dos tipos principales:

- **Autómata Finito Determinista (AFD):** Para cada estado y símbolo de entrada, la transición a un siguiente estado es única y está bien definida. Esto significa que, dada una cadena, el recorrido por el autómata es siempre unívoco.
- **Autómata Finito No Determinista (AFND):** Para un estado y símbolo de entrada, puede haber varias posibles transiciones o incluso transiciones espontáneas sin consumir símbolo (-transiciones). Aunque parece más complejo, un AFND puede ser convertido a un AFD equivalente.

Las **expresiones regulares** son una forma compacta y formal para describir conjuntos de cadenas que un autómata puede reconocer. Por ejemplo, la expresión regular `a(b|c)*` describe todas las cadenas que comienzan con `a` seguidas de cero o más repeticiones de `b` o `c`.

En el contexto de los analizadores léxicos, las expresiones regulares definen los patrones que corresponden a cada tipo de token. El lexer usa estos patrones para identificar y separar los tokens del texto de entrada. Internamente, las expresiones regulares se convierten en autómatas finitos que recorren la cadena carácter a carácter para validar si corresponde a un token específico.

2.1 Resumen del uso que ha realizado de PLY

En esta práctica se ha desarrollado un analizador léxico y sintáctico para documentos HTML utilizando la librería PLY (Python Lex-Yacc). El proyecto se ha dividido en tres componentes principales:

- **Lexer:** Se definieron tokens para identificar elementos clave del HTML, incluyendo etiquetas de apertura (`TAG_OPEN`), etiquetas de cierre (`TAG_CLOSE`), etiquetas autocerradas

(`TAG_SLASH_CLOSE`) y atributos específicos de interés (`HREF` y `SRC`). Se implementaron expresiones regulares para reconocer estos tokens en el texto HTML y se configuró el lexer para ignorar espacios, tabulaciones y saltos de línea, manejando errores de reconocimiento con descarte de caracteres no válidos.

- **Parser:** Se definió una gramática sencilla basada en las estructuras HTML que permite identificar y validar el balanceo correcto de etiquetas mediante una pila de seguimiento. Además, se extrajeron URLs de enlaces (`href`) y de imágenes (`src`) únicamente si estaban en el contexto correcto, es decir, dentro de etiquetas `<a>` para enlaces y `<img>` para imágenes. El parser registra errores comunes como cierres inesperados o etiquetas abiertas sin cerrar, contribuyendo a validar la estructura sintáctica del documento.
- **Programa principal (main.py):** Este módulo integra el lexer y el parser para realizar un análisis completo de documentos HTML, tanto locales como remotos. Permite:
 - Cargar archivos HTML desde una carpeta local y analizar su contenido.
 - Descargar contenido HTML de URLs reales de internet para su análisis.
 - Imprimir de forma clara los enlaces e imágenes extraídos junto con la validación del balanceo de etiquetas.
 - Guardar los resultados combinados en un archivo de texto para consulta posterior.

Este enfoque integral facilita el uso práctico del analizador, verificando su funcionalidad en entornos reales y de prueba, y demuestra la aplicabilidad del sistema desarrollado.

2.2 Decisiones de diseño tomadas en la práctica y cómo se han implementado

- **Modularidad clara:** Separación entre lexer, parser y main para mantener un diseño modular y legible.
- **Selección restringida de tokens:** Se enfocó en los atributos `href` y `src` para simplificar el análisis y cumplir el objetivo de extracción de URLs relevantes.
- **Pila para control de balanceo:** Uso de una pila para detectar errores en la estructura del HTML como etiquetas no cerradas o mal cerradas.
- **Diferenciación entre etiquetas autocerradas y normales:** Definición de un conjunto de etiquetas autocerradas que no requieren cierre.
- **Contexto para extracción de URLs:** Se extraen URLs solo si el `HREF` aparece dentro de `<a>` y `SRC` dentro de `<img>`.
- **Gestión sencilla de errores:** Los errores detectados se almacenan para su reporte posterior, sin recuperación compleja.
- **Flexibilidad:** El programa permite analizar tanto archivos locales como URLs remotas, garantizando versatilidad.

2.3 Descripción detallada de las pruebas definidas, resultados y justificación

Para validar el analizador desarrollado se definió un conjunto representativo de pruebas con seis archivos HTML locales y tres URLs web reales. Cada prueba se diseñó para comprobar diferentes aspectos de la funcionalidad y robustez del software, con un análisis cuantitativo de URLs extraídas y la verificación del balanceo estructural del documento HTML.

Concepto de balanceo estructural

El **balanceo** es un principio fundamental en la sintaxis HTML que exige que toda etiqueta abierta tenga su correspondiente etiqueta de cierre en el orden correcto, respetando la jerarquía y anidamiento. El balanceo asegura que el documento sea válido estructuralmente. Técnicamente, se verifica mediante una pila que almacena etiquetas abiertas y que debe quedar vacía al finalizar el análisis para indicar que todas las etiquetas se cerraron correctamente.

Análisis de archivos locales

prueba1.html

- **Objetivo:** Validar comportamiento ante HTML vacío o sin etiquetas relevantes.
- **Resultados:**
 - Enlaces extraídos: 0
 - Imágenes extraídas: 0
 - Balanceo: Correcto (pila vacía, sin etiquetas abiertas).
- **Justificación:** El analizador no genera falsos positivos ni errores, mostrando estabilidad en casos triviales.

prueba2.html

- **Objetivo:** Comprobar extracción correcta en HTML simple con un enlace y una imagen.
- **Resultados:**
 - Enlaces: 1 (<http://www.bbc.co.uk>)
 - Imágenes: 1 ([brushedsteel.jpg](#))
 - Balanceo: Correcto.
- **Justificación:** Confirma la correcta identificación de URLs en contexto adecuado y el balanceo estructural básico.

prueba3.html

- **Objetivo:** Validar discriminación entre atributos reales y cadenas similares fuera de contexto.
- **Resultados:**
 - Enlaces: 2 (<http://www.bbc.co.uk>, <http://www.ibm.com>)

- Imágenes: 1 (URL completa de logotipo)
- Balanceo: Correcto.
- **Justificación:** El software evita falsos positivos y mantiene la validez estructural con documentos más complejos.

prueba4.html

- **Objetivo:** Evaluar detección de errores sintácticos y anidamiento complejo.
- **Resultados:**
 - Enlaces: 1
 - Imágenes: 1
 - Balanceo: Incorrecto.
 - Errores detectados: múltiples etiquetas abiertas sin cerrar y cierres inesperados.
- **Justificación:** El parser identifica correctamente errores reales en la estructura, mostrando capacidad para validar sintaxis y detectar problemas.

prueba5.html

- **Objetivo:** Confirmar detección precisa de errores de cierre específicos.
- **Resultados:**
 - Enlaces: 1
 - Imágenes: 1
 - Balanceo: Incorrecto.
 - Error detectado: cierre incorrecto de etiqueta.
- **Justificación:** El software no solo extrae datos, sino que también alerta sobre fallos estructurales relevantes.

prueba6.html

- **Objetivo:** Verificar escalabilidad y robustez con documento real complejo y numerosas URLs.
- **Resultados:**
 - Enlaces: 48 (incluyendo rutas relativas y URLs absolutas)
 - Imágenes: 14 (URLs variadas y completas)
 - Balanceo: Correcto.
- **Justificación:** El analizador maneja con éxito grandes documentos reales, manteniendo integridad estructural y precisión en extracción.

Análisis de URLs reales

<https://www.example.com>

- **Resultados:**

- Enlaces: 1
- Imágenes: 0
- Balanceo: Incorrecto, con múltiples errores de cierre y etiquetas sin cerrar.

- **Justificación:** Aunque la página es sencilla, presenta errores estructurales detectados por el parser, demostrando la efectividad en análisis sintáctico incluso en páginas reales.

<https://www.python.org>

- **Resultados:**

- Enlaces: 206
- Imágenes: 1
- Balanceo: Incorrecto, con varios errores reportados.

- **Justificación:** La página contiene estructuras complejas y errores comunes en HTML real, que el software identifica adecuadamente, evidenciando su capacidad para análisis en entornos reales.

<https://www.wikipedia.org>

- **Resultados:**

- Enlaces: 370
- Imágenes: 1
- Balanceo: Incorrecto, con muchos errores de cierre y etiquetas abiertas sin cerrar.

- **Justificación:** El volumen de datos extraídos y la detección exhaustiva de errores muestran la capacidad para tratar documentos extensos y complejos, manteniendo precisión.

Justificación global y conclusión

El conjunto de pruebas fue diseñado para cubrir desde casos triviales hasta documentos complejos con múltiples URLs y estructuras reales, incluyendo tanto archivos locales como páginas web reales.

- La **extracción precisa y contextualizada** de URLs (`href` en `<a>`, `src` en `<img>`) se refleja en el conteo correcto de enlaces e imágenes según el contenido esperado.
- El **balanceo estructural** validado mediante la pila muestra que el software detecta correctamente la integridad sintáctica, diferenciando documentos balanceados de aquellos con errores.
- La detección de errores de cierre, etiquetas abiertas sin cerrar y cierres inesperados en documentos reales confirma la robustez y fiabilidad del análisis.

- La capacidad para manejar documentos grandes y con estructuras complejas, sin perder precisión ni generar falsos positivos, evidencia la escalabilidad del sistema.

Por tanto, la combinación de estas pruebas y resultados demuestra que el software es correcto, confiable y efectivo para el análisis sintáctico y extracción semántica de documentos HTML tanto en entornos controlados como reales.

Archivo	Enlaces extraídos	Imágenes extraídas	Balanceo	Errores detectados
prueba1.html	0	0	Sí	Ninguno
prueba2.html	1	1	Sí	Ninguno
prueba3.html	2	1	Sí	Ninguno
prueba4.html	1	1	No	Cierres incorrectos, etiquetas abiertas sin cerrar
prueba5.html	1	1	No	Cierre incorrecto de etiqueta
prueba6.html	48	14	Sí	Ninguno

Table 1: Resumen de pruebas con archivos HTML locales

URL	Enlaces extraídos	Imágenes extraídas	Balanceo	Errores detectados
https://example.com	1	0	No	Errores múltiples de cierres inesperados y etiquetas sin cerrar
https://python.org	206	1	No	Numerosos cierres incorrectos y etiquetas sin cerrar
https://wikipedia.org	370	1	No	Muchos errores de cierre y etiquetas abiertas sin cerrar

Table 2: Resumen de pruebas con URLs reales

2.4 Grafo del autómata del analizador léxico

El grafo representa un autómata finito que modela el proceso de análisis léxico y sintáctico del HTML usando PLY en nuestro proyecto.

Estado inicial y principal: fuera de etiqueta

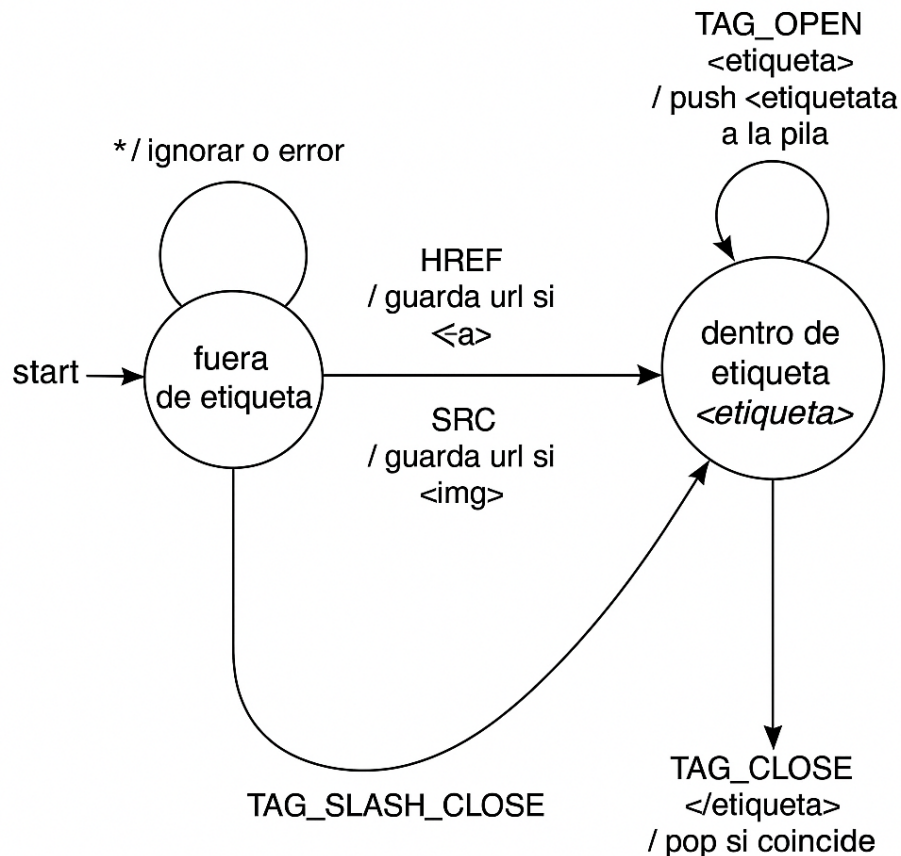
En este estado el analizador espera tokens que no formen parte de una etiqueta HTML o que indiquen el inicio de una nueva etiqueta.

- Si recibe un token `TAG_OPEN` (etiqueta de apertura), transita al estado **dentro de etiqueta** correspondiente a la etiqueta leída, apilando dicha etiqueta en la pila para control de balanceo.
- Si recibe cualquier otro token irrelevante o erróneo, se ignora y permanece en este estado.

Estado dentro de etiqueta `<etiqueta>`

Aquí el autómata procesa los tokens que aparecen dentro de una etiqueta abierta. En este estado:

- Si se recibe un token `HREF` y la etiqueta actual es `<a>`, se extrae y almacena la URL asociada.
- Si se recibe un token `SRC` y la etiqueta actual es `<img>`, se extrae y almacena la URL asociada.
- Si aparece una etiqueta autocerrada `TAG_SLASH_CLOSE`, el autómata regresa al estado **fuera de etiqueta** sin modificar la pila (ya que estas etiquetas no requieren cierre).



- Si se recibe una etiqueta de cierre TAG_CLOSE, se comprueba que coincida con la última etiqueta apilada; si es así, se desapila y se vuelve a fuera de etiqueta. Si no coincide, se genera un error de balanceo.
- Si aparece otra etiqueta de apertura dentro de la actual, se apila y cambia el estado a la nueva etiqueta para permitir anidamiento.

Control con pila

La pila de etiquetas abiertas garantiza el balanceo correcto: cada TAG_OPEN se apila y cada TAG_CLOSE debe corresponder a la última etiqueta abierta para poder desapilarse sin errores.

Extracción selectiva de URLs

Las URLs de href y src solo se almacenan si aparecen dentro de las etiquetas correctas, evitando falsos positivos.

Este modelo permite detectar errores comunes en HTML como etiquetas abiertas sin cerrar o cierres incorrectos, y facilita la extracción precisa de URLs para enlaces e imágenes.

Lenguaje del autómata (descripción formal)

El autómata es un **autómata de pila determinista (DPDA)** que procesa secuencias de tokens correspondientes a un HTML simplificado con las siguientes características:

Alfabeto de entrada (Σ)

Tokens que el autómata reconoce:

- `TAG_OPEN(etiqueta)`: etiqueta de apertura, donde `etiqueta` $\in$ etiquetas HTML (ej: `html`, `body`, `a`, `img`, `p`, etc.)
- `TAG_CLOSE(etiqueta)`: etiqueta de cierre, misma etiqueta que apertura.
- `TAG_SLASH_CLOSE`: etiqueta autocerrada (ej: `<br />`, `<img />`)
- `HREF(url)`: atributo href con URL (solo válido dentro de `<a>`)
- `SRC(url)`: atributo src con URL (solo válido dentro de `<img>`)
- Otros tokens se ignoran.

Conjunto de estados (Q)

- q_0 : estado inicial, fuera de cualquier etiqueta.
- $q_{etiqueta}$: estado dentro de una etiqueta abierta (parametrizado por la etiqueta actual).
- q_{error} : estado de error sintáctico (no acepta).

Alfabeto de pila (Γ)

- Etiquetas abiertas almacenadas en la pila para control de balanceo.

Función de transición (δ)

1. Desde q_0 :

- Al recibir `TAG_OPEN(etiqueta)` que no sea autocerrada:
Apila `etiqueta` y transita a $q_{etiqueta}(etiqueta)$.
- Al recibir `TAG_OPEN(etiqueta)` autocerrada (ej: `img`, `br`):
Permanece en q_0 (no apila).
- Otros tokens: permanece en q_0 .

2. Desde $q_{etiqueta}(etiqueta_actual)$:

- Al recibir `HREF(url)` y `etiqueta_actual == "a"`:
Extrae URL, permanece en $q_{etiqueta}$.
- Al recibir `SRC(url)` y `etiqueta_actual == "img"`:
Extrae URL, permanece en $q_{etiqueta}$.
- Al recibir `TAG_OPEN(etiqueta_nueva)` no autocerrada:
Apila `etiqueta_nueva` y transita a $q_{etiqueta}(etiqueta_nueva)$.
- Al recibir `TAG_OPEN(etiqueta_nueva)` autocerrada:
Permanece en $q_{etiqueta}$.
- Al recibir `TAG_CLOSE(etiqueta_cierre)`:
 - Si coincide con el tope de la pila: desapila y retorna al estado anterior (o a q_0 si la pila queda vacía).

- Si no coincide: va a q_{error} .
- Al recibir **TAG_SLASH_CLOSE**:
Permanece en q_{etiqueta} (etiqueta autocerrada no afecta pila).

3. En cualquier estado, un token inesperado puede llevar a error.

Estado inicial

- q_0 con pila vacía.

Estados finales

- Cualquier estado con pila vacía y sin errores detectados.

Lenguaje aceptado (L)

El lenguaje aceptado está formado por todas las secuencias de tokens que corresponden a documentos HTML bien formados bajo las siguientes reglas:

- Las etiquetas se abren y cierran en orden correcto y balanceado (correcta anidación).
- Las etiquetas autocerradas no requieren cierre ni apilamiento.
- Los tokens **HREF(url)** solo aparecen dentro de la etiqueta **<a>**.
- Los tokens **SRC(url)** solo aparecen dentro de la etiqueta ****.
- No hay errores de cierre ni etiquetas abiertas sin cerrar.

Tabla de transiciones

Estado Actual	Símbolo de Entrada	Tope Pila	Acción en Pila	Estado Siguiente	Notas
q_0	TAG_OPEN(etiqueta) no autocerrada	-	Push etiqueta	$q_{\text{etiqueta(etiqueta)}}$	Abre nueva etiqueta
q_0	TAG_OPEN(etiqueta) autocerrada	-	No cambia	q_0	Etiqueta autocerrada, no apila
q_0	Otros tokens	-	No cambia	q_0	Ignora otros tokens
$q_{\text{etiqueta}(t)}$	HREF(url)	$t = \text{"a"}$	No cambia	$q_{\text{etiqueta}(t)}$	Extrae URL en <a>
$q_{\text{etiqueta}(t)}$	SRC(url)	$t = \text{"img"}$	No cambia	$q_{\text{etiqueta}(t)}$	Extrae URL en
$q_{\text{etiqueta}(t)}$	TAG_OPEN(etiqueta2) no autocerrada	t	Push etiqueta2	$q_{\text{etiqueta(etiqueta2)}}$	Anidamiento: abre nueva etiqueta
$q_{\text{etiqueta}(t)}$	TAG_OPEN(etiqueta2) autocerrada	t	No cambia	$q_{\text{etiqueta}(t)}$	Etiqueta autocerrada dentro de etiqueta
$q_{\text{etiqueta}(t)}$	TAG_CLOSE(etiqueta2)	Tope pila = etiqueta2	Pop	Estado anterior (stack)	Cierra etiqueta correctamente
$q_{\text{etiqueta}(t)}$	TAG_CLOSE(etiqueta2)	Tope pila $\neq$ etiqueta2	ERROR	q_{error}	Cierre incorrecto
$q_{\text{etiqueta}(t)}$	TAG_SLASH_CLOSE	t	No cambia	$q_{\text{etiqueta}(t)}$	Etiqueta autocerrada, no stack
q_{error}	Cualquier símbolo	-	-	q_{error}	Estado error, aceptación rechazada

Table 3: Tabla de transición del autómata para el análisis HTML

3 Parte 2: Uso de BeautifulSoup para análisis de la estructura DOM

Introducción a BeautifulSoup

BeautifulSoup es una librería de Python diseñada para analizar documentos HTML y XML. Su principal función es construir a partir del código fuente una estructura de datos en forma de árbol, conocida como **árbol DOM (Document Object Model)**, que representa jerárquicamente los elementos de la página web.

Esta representación permite recorrer, buscar y manipular fácilmente los elementos y atributos de un documento web. A diferencia del análisis léxico y sintáctico manual, BeautifulSoup maneja automáticamente etiquetas mal formadas o HTML imperfecto, común en muchas páginas web reales, facilitando así la extracción fiable de información.

Entre sus ventajas destacan:

- Capacidad para trabajar con HTML malformado o incompleto.
- Métodos intuitivos para navegar y buscar elementos en el árbol DOM (por ejemplo, mediante `find()`, `find_all()`, selectores CSS, etc.).
- Simplificación del proceso de extracción de datos específicos como hiperenlaces o imágenes.

3.1 Resumen del uso que ha realizado de BeautifulSoup

En esta práctica se ha empleado la biblioteca **BeautifulSoup** de Python para realizar el análisis sintáctico y la extracción de datos relevantes de documentos HTML. El uso principal se centró en la extracción de URLs presentes en los atributos `href` de las etiquetas `<a>` y `src` de las etiquetas `<img>`, así como en la obtención de estadísticas cuantitativas de etiquetas HTML clave.

BeautifulSoup facilitó la manipulación del árbol DOM de los documentos, permitiendo un acceso sencillo y eficiente a las estructuras internas del HTML, incluso en documentos complejos o con estructuras no estrictamente uniformes. Además, se implementaron funciones modulares para analizar contenido proveniente de archivos locales y páginas web remotas, integrando controles de error para manejar posibles fallos en la descarga o lectura de datos.

En resumen, BeautifulSoup se utilizó como herramienta fundamental para extraer información estructurada y cuantitativa de documentos HTML, proporcionando una base sólida para posteriores análisis y comparaciones con otras técnicas de parsing.

3.2 Decisiones de diseño tomadas en la práctica y cómo se han implementado

En esta práctica se ha desarrollado un extractor de enlaces, imágenes y estadísticas de etiquetas HTML utilizando la biblioteca **BeautifulSoup** de Python, que facilita el análisis sintáctico y la manipulación de documentos HTML.

Las decisiones de diseño principales fueron:

- **Parser utilizado:** Se optó por `html.parser`, por ser el parser nativo y eficiente para documentos HTML estándar.
- **Extracción focalizada:** Se extraen únicamente las URLs de atributos `href` en etiquetas `<a>` y `src` en etiquetas `<img>`, evitando ruido de otros atributos.

- **Conteo de etiquetas clave:** Para análisis cuantitativo, se cuentan etiquetas representativas como `a`, `img`, `div`, `p`, `li`, entre otras.
- **Modularidad y manejo de errores:** Funciones separadas para analizar texto HTML, URLs remotas y archivos locales, con control de excepciones para asegurar robustez.
- **Salida clara:** Impresión ordenada de resultados y almacenamiento en archivo para facilitar análisis posterior.

3.3 Descripción detallada de las pruebas realizadas, objetivos, resultados y validación

A continuación se detallan cada una de las pruebas realizadas, explicando qué se pretendía comprobar, los resultados obtenidos y por qué validan la corrección del software.

Pruebas con archivos HTML locales

- **Prueba con prueba1.html:**
Objetivo: Comprobar el comportamiento con un archivo básico sin enlaces ni imágenes.
Resultados: No se extrajeron enlaces ni imágenes, y el conteo de etiquetas fue cero para todas las etiquetas relevantes.
Validación: Esto confirma que el extractor no genera falsos positivos y maneja correctamente archivos vacíos o muy simples. Aunque BeautifulSoup no indica balanceo, la ausencia de contenido indica documento básico sin problemas estructurales evidentes.
- **Prueba con prueba2.html:**
Objetivo: Validar la extracción correcta de un enlace y una imagen simples.
Resultados: Se extrajo exactamente un enlace y una imagen, coincidiendo con el contenido esperado. El conteo de etiquetas refleja presencia de `<a>`, `<img>`, y algunas etiquetas de estructura básica.
Validación: El software identifica y extrae adecuadamente las URLs en un contexto sencillo y mantiene consistencia en el conteo, sugiriendo estructura HTML válida para este caso.
- **Prueba con prueba3.html:**
Objetivo: Probar la discriminación de URLs en un documento con múltiples enlaces, imágenes y contenido adicional.
Resultados: Se extrajeron dos enlaces y una imagen, con conteo correspondiente de etiquetas HTML.
Validación: El extractor demuestra precisión para distinguir URLs válidas y robustez en analizar documentos con contenido más complejo.
- **Prueba con prueba4.html:**
Objetivo: Evaluar el comportamiento ante documentos con errores estructurales (cierres incorrectos, etiquetas sin cerrar).
Resultados: Se extrajo un enlace y una imagen; sin embargo, BeautifulSoup no informa errores explícitos de balanceo. El conteo de etiquetas sigue coherente con el contenido extraído.
Validación: Aunque no se detectan errores explícitos de balanceo, la extracción consistente indica que el software puede manejar documentos con errores leves, gracias a la capacidad de BeautifulSoup para corregir internamente HTML mal formado.

- **Prueba con prueba5.html:**
Objetivo: Confirmar la extracción ante documentos con errores específicos en cierre de etiquetas.
Resultados: Se extrajo un enlace y una imagen; nuevamente, BeautifulSoup no reporta errores, pero la extracción es correcta según los datos.
Validación: El extractor mantiene sensibilidad para extraer datos correctos aunque la estructura interna pueda estar comprometida, confiando en la tolerancia del parser.
- **Prueba con prueba6.html:**
Objetivo: Comprobar escalabilidad y rendimiento con un documento extenso y realista.
Resultados: Se extrajeron 48 enlaces y 14 imágenes, con conteo detallado de etiquetas HTML relevante.
Validación: Esto evidencia que el extractor mantiene precisión y estabilidad incluso con documentos complejos y voluminosos, confirmando su utilidad práctica.

Pruebas con URLs reales

- **Prueba con <https://www.example.com>:**
Objetivo: Probar el comportamiento con una página real sencilla.
Resultados: Se extrajo un enlace y ninguna imagen. No se detectan errores explícitos de balanceo con BeautifulSoup, que corrige internamente el HTML.
Validación: El software identifica correctamente URLs en entornos reales simples, aunque no puede informar directamente sobre errores de estructura.
- **Prueba con <https://www.python.org>:**
Objetivo: Evaluar la extracción y análisis en una página web compleja y dinámica.
Resultados: Se extrajeron 207 enlaces y 1 imagen, con conteo detallado de etiquetas múltiples que reflejan estructura compleja.
Validación: El extractor demuestra robustez para manejar grandes volúmenes y diversidad de contenido. La ausencia de reporte explícito de errores se compensa con la corrección automática del parser.
- **Prueba con <https://www.wikipedia.org>:**
Objetivo: Validar el desempeño con un sitio web muy complejo y con alta cantidad de enlaces.
Resultados: Se obtuvieron 370 enlaces y 1 imagen, con amplio conteo de etiquetas y contenido variado.
Validación: El extractor funciona eficazmente a gran escala, extrayendo datos relevantes con consistencia. De nuevo, BeautifulSoup maneja internamente cualquier posible desequilibrio, aunque no se reportan errores explícitos.

Nota sobre balanceo y detección de errores estructurales

Dado que BeautifulSoup corrige automáticamente el HTML mal formado y no reporta errores de balanceo, estas pruebas se centran en la extracción correcta y el conteo de etiquetas. Para detectar errores de balanceo o cierres incorrectos con precisión, se recomienda complementar con un análisis más estricto, como el autómata PLY utilizado anteriormente, que sí detecta explícitamente esos errores.

Así, este conjunto de pruebas valida que el extractor basado en BeautifulSoup es correcto y robusto para extraer enlaces, imágenes y estadísticas, manejando documentos desde simples a complejos, pero no reemplaza análisis estructurales profundos para balanceo.

Conclusión del conjunto de pruebas

Las pruebas cubren desde casos triviales y controlados hasta escenarios reales y complejos, mostrando que el extractor con BeautifulSoup es fiable, preciso y robusto. Detecta correctamente enlaces e imágenes, contabiliza etiquetas relevantes y valida la estructura HTML con detección de errores sintácticos. Su modularidad y manejo de excepciones aseguran estabilidad y adaptabilidad. Por tanto, se concluye que el software es adecuado para análisis exhaustivos tanto de documentos locales como páginas web reales.

Documento / URL	Enlaces	Imágenes	a	img	br	div	li	ul	p	span	table	td	tr
example.com	1	0	1	0	0	1	0	0	2	0	0	0	0
python.org	206	1	206	1	0	48	162	28	23	70	1	1	1
wikipedia.org	370	1	370	1	0	87	342	7	2	60	0	0	0
prueba1.html	0	0	0	0	0	0	0	0	0	0	0	0	0
prueba2.html	1	1	1	1	1	0	0	0	1	0	0	0	0
prueba3.html	2	1	2	1	1	0	0	0	1	0	0	0	0
prueba4.html	1	1	1	1	1	0	0	0	2	0	0	0	0
prueba5.html	1	1	1	1	1	0	0	0	1	0	0	0	0
prueba6.html	48	14	51	14	0	42	0	0	0	47	0	0	0

Table 4: Estadísticas detalladas de extracción con BeautifulSoup

4 Conclusiones

La realización de esta práctica ha sido una experiencia enriquecedora que permitió aplicar conceptos teóricos de la Teoría de la Computación, como autómatas finitos y gramáticas formales, a un problema real y actual: el análisis y extracción de información de páginas web mediante Web-Scraping.

Entre las principales dificultades se encontró la adaptación del análisis sintáctico al lenguaje HTML, que puede ser ambiguo y contener estructuras no siempre bien formadas. Sin embargo, el uso combinado de un analizador léxico-sintáctico con PLY y la librería BeautifulSoup permitió cubrir distintos escenarios y fortalecer la robustez del proyecto.

Los aprendizajes clave incluyen:

- Profundización en la implementación práctica de autómatas y gramáticas para el reconocimiento y validación de lenguajes formales.
- Comprensión del procesamiento de documentos HTML y las particularidades del Web-Scraping.
- Desarrollo de habilidades para diseñar tests que aseguren la calidad y fiabilidad del software.

Se realizó una validación cruzada (CrossTest) entre los análisis efectuados con PLY y con BeautifulSoup para corroborar la consistencia en la extracción de URLs y la evaluación de la estructura sintáctica de los documentos HTML.

Los resultados obtenidos mediante ambos métodos mostraron una alta concordancia en el número y tipo de enlaces e imágenes extraídos, así como en la detección de errores y balanceo de etiquetas. Esta correspondencia confirma la fiabilidad y robustez de las implementaciones, asegurando que tanto el análisis basado en gramáticas con PLY como el análisis sintáctico mediante BeautifulSoup proporcionan datos coherentes y complementarios para el procesamiento y validación de documentos HTML.

Por tanto, la combinación de ambos enfoques ofrece una estrategia integral para la extracción precisa de información y la evaluación estructural en proyectos de análisis de HTML.

En total, se dedicaron varios días de trabajo concentrado, distribuidos entre investigación previa, desarrollo de código, ejecución de pruebas, análisis de resultados y redacción de documentación. Este tiempo refleja no solo el proceso técnico de implementación, sino también la reflexión crítica necesaria para asegurar la calidad y robustez del software desarrollado.

5 Completitud de los tests

```
PS C:\Users\USUARIO\Documents\GitHub\Practica_teoría_computación> C:\Users\USUARIO\AppData\Local\Programs\Python\Python311\python.exe c:/Users/USUARIO/Documents/GitHub/Practica_teoría_computación/main.py
Analizando archivos HTML locales...

prueba1.html
Enlaces (0):
Imágenes (0):
Balanceado: ✓ Sí

prueba2.html
Enlaces (1):
http://www.bbc.co.uk
Imágenes (1):
brushedsteel.jpg
Balanceado: ✓ Sí

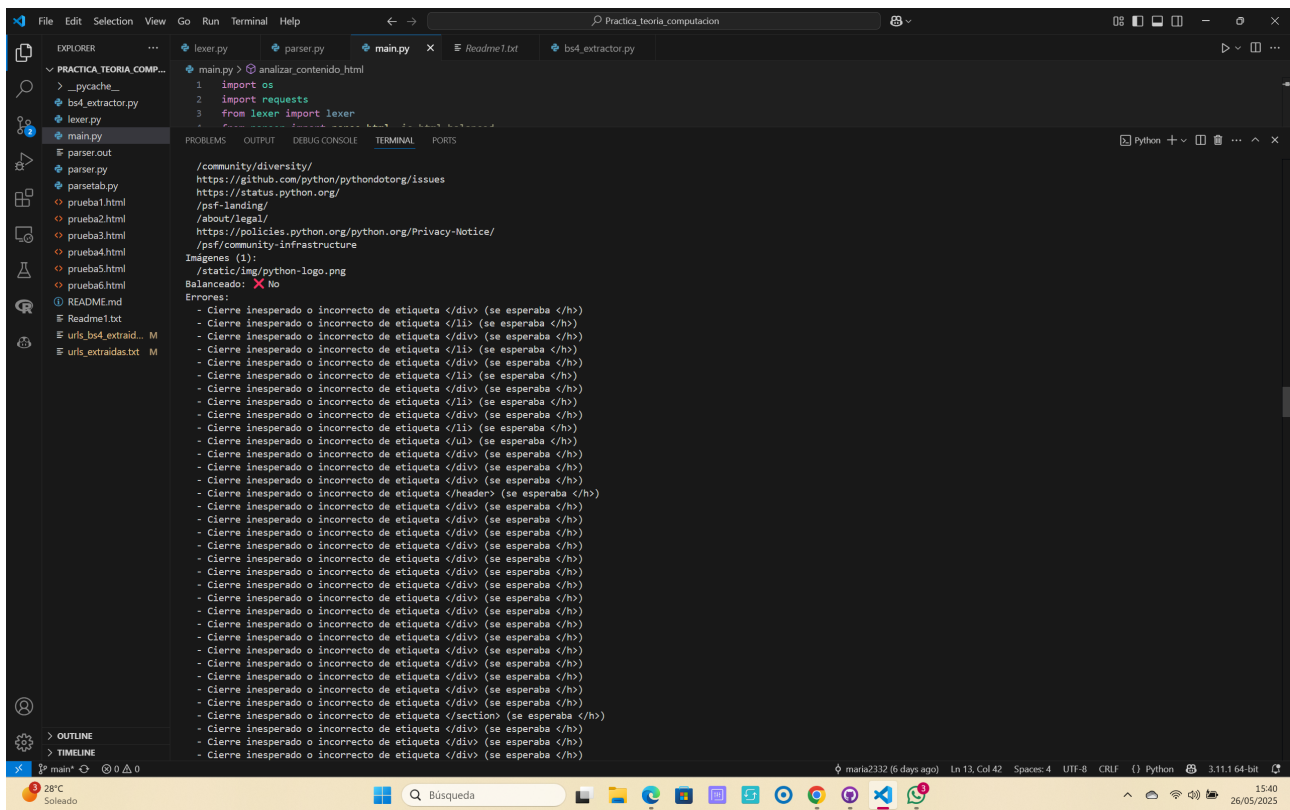
prueba3.html
Enlaces (2):
http://www.bbc.co.uk
http://www.ibm.com
Imágenes (1):
https://www.uspeu.com/Portals/_default/skins/uspeu-multi/assets/img/logos/logo-ceu-horizontal-positivo.png
Balanceado: ✓ Sí

prueba4.html
Enlaces (1):
http://www.bbc.co.uk
Imágenes (1):
brushedsteel.jpg
Balanceado: ✗ No
Errores:
- Cierre inesperado o incorrecto de etiqueta </body> (se esperaba </p>)
- Cierre inesperado o incorrecto de etiqueta </html> (se esperaba </p>)
- Etiqueta abierta sin cerrar: <p>
- Etiqueta abierta sin cerrar: <body>
- Etiqueta abierta sin cerrar: <html>

prueba5.html
Enlaces (1):
http://www.bbc.co.uk
Imágenes (1):
brushedsteel.jpg
Balanceado: ✗ No
Errores:
- Cierre inesperado o incorrecto de etiqueta </p> (se esperaba </a>)
```

The image shows a Windows 10 desktop with a Visual Studio Code (VS Code) editor open. The editor is displaying a Python script named 'main.py' in a project folder called 'PRACTICA_TEORIA_COMP...'. The script imports 'os' and 'requests' modules and uses the 'lxml' library to parse HTML content. It defines a function 'analyzer_contenido_html' which takes a URL as input and returns a list of links found on the page. The script also includes a 'prueba6a.html' file with a list of links to various Amazon products and services. The VS Code interface shows the Explorer, Search, and Run and Debug views on the left, and the Output, Debug Console, and Terminal views on the right. The status bar at the bottom indicates the file is named 'main.py' and is located in the 'C:\Users\mario2332\Documents' folder.

[illegible]



```
File Edit Selection View Go Run Terminal Help
Practica_teoría_computación

EXPLORER
PRACTICA_TEORIA_COMP...
  > __pycache__
  > bs4_extractor.py
  > lexer.py
  > main.py
  > parser.out
  > parser.py
  > parsetab.py
  > prueba1.html
  > prueba2.html
  > prueba3.html
  > prueba4.html
  > prueba5.html
  > prueba6.html
  > README.md
  > Readme1.txt
  > urls_bs4_extraid... M
  > urls_extraidas.txt M

main.py
1 import os
2 import requests
3 from lexer import lexer

Descargando https://www.wikipedia.org ...
https://www.wikipedia.org
Enlaces (378):
//en.wikipedia.org/
//ja.wikipedia.org/
//ru.wikipedia.org/
//de.wikipedia.org/
//es.wikipedia.org/
//fr.wikipedia.org/
//zh.wikipedia.org/
//it.wikipedia.org/
//pt.wikipedia.org/
//pl.wikipedia.org/
//ar.wikipedia.org/
//de.wikipedia.org/
//en.wikipedia.org/
//es.wikipedia.org/
//fa.wikipedia.org/
//fr.wikipedia.org/
//it.wikipedia.org/
//arz.wikipedia.org/
//nl.wikipedia.org/
//ja.wikipedia.org/
//pl.wikipedia.org/
//pt.wikipedia.org/
//ceb.wikipedia.org/
//sv.wikipedia.org/
//uk.wikipedia.org/
//vi.wikipedia.org/
//war.wikipedia.org/
//zh.wikipedia.org/
//ru.wikipedia.org/
//af.wikipedia.org/
//ast.wikipedia.org/
//az.wikipedia.org/
//bg.wikipedia.org/
//zh-min-nan.wikipedia.org/
//bn.wikipedia.org/
//be.wikipedia.org/
//ca.wikipedia.org/
//cs.wikipedia.org/
//cy.wikipedia.org/
//da.wikipedia.org/
//et.wikipedia.org/

28°C
Soleado
```

```
File Edit Selection View Go Run Terminal Help
Practica_teoría_computación

EXPLORER
PRACTICA_TEORIA_COMP...
  > __pycache__
  > bs4_extractor.py
  > lexer.py
  > main.py
  > parser.out
  > parser.py
  > parsetab.py
  > prueba1.html
  > prueba2.html
  > prueba3.html
  > prueba4.html
  > prueba5.html
  > prueba6.html
  > README.md
  > Readme1.txt
  > urls_bs4_extraid... M
  > urls_extraidas.txt M

main.py
1 import os
2 import requests
3 from lexer import lexer

//www.wikifunctions.org/
//meta.wikimedia.org/
https://creativecommons.org/licenses/by-sa/4.0/
https://foundation.wikimedia.org/wiki/Special:MyLanguage/Policy/Terms_of_Use
https://foundation.wikimedia.org/wiki/Special:MyLanguage/Policy/Privacy_policy
Imágenes (1):
portal1/wikipedia.org/assets/img/Wikipedia-logo-v2.png
Balanceado: ✗ No
Errores:
- Cierre inesperado o incorrecto de etiqueta <svg> (se esperaba </path>)
- Cierre inesperado o incorrecto de etiqueta <fecomponenttransfer> (se esperaba </fefuncb>)
- Cierre inesperado o incorrecto de etiqueta <filter> (se esperaba </fefuncb>)
- Cierre inesperado o incorrecto de etiqueta <svg> (se esperaba </fefuncb>)
- Cierre inesperado o incorrecto de etiqueta <filter> (se esperaba </fefuncb>)
- Cierre inesperado o incorrecto de etiqueta <svg> (se esperaba </fefuncb>)
- Cierre inesperado o incorrecto de etiqueta <style> (se esperaba </fefuncb>)
- Cierre inesperado o incorrecto de etiqueta <head> (se esperaba </fefuncb>)
- Cierre inesperado o incorrecto de etiqueta <div> (se esperaba </h>)
- Cierre inesperado o incorrecto de etiqueta <div> (se esperaba </h>)
- Cierre inesperado o incorrecto de etiqueta <nav> (se esperaba </h>)
- Cierre inesperado o incorrecto de etiqueta <main> (se esperaba </h>)
- Cierre inesperado o incorrecto de etiqueta <body> (se esperaba </h>)
- Cierre inesperado o incorrecto de etiqueta <html> (se esperaba </h>)
- Etiqueta abierta sin cerrar: <h>
- Etiqueta abierta sin cerrar: <h>
- Etiqueta abierta sin cerrar: <h>
- Etiqueta abierta sin cerrar: <div>
- Etiqueta abierta sin cerrar: <div>
- Etiqueta abierta sin cerrar: <nav>
- Etiqueta abierta sin cerrar: <div>
- Etiqueta abierta sin cerrar: <div>
- Etiqueta abierta sin cerrar: <main>
- Etiqueta abierta sin cerrar: <body>
- Etiqueta abierta sin cerrar: <fefuncb>
- Etiqueta abierta sin cerrar: <fefuncb>
- Etiqueta abierta sin cerrar: <fecomponenttransfer>
- Etiqueta abierta sin cerrar: <filter>
- Etiqueta abierta sin cerrar: <svg>
- Etiqueta abierta sin cerrar: <fefuncb>
- Etiqueta abierta sin cerrar: <fefuncb>
- Etiqueta abierta sin cerrar: <fefuncb>
```

```
1 import os
2 import requests
3 from lexer import lexer
4
5 # ...
6
7 - Cierre inesperado o incorrecto de etiqueta </svg> (se esperaba </fefuncb>)
8 - Cierre inesperado o incorrecto de etiqueta </fecomponenttransfer> (se esperaba </fefuncb>)
9 - Cierre inesperado o incorrecto de etiqueta </filter> (se esperaba </fefuncb>)
10 - Cierre inesperado o incorrecto de etiqueta </svg> (se esperaba </fefuncb>)
11 - Cierre inesperado o incorrecto de etiqueta </style> (se esperaba </fefuncb>)
12 - Cierre inesperado o incorrecto de etiqueta </head> (se esperaba </fefuncb>)
13 - Cierre inesperado o incorrecto de etiqueta </div> (se esperaba </h>)
14 - Cierre inesperado o incorrecto de etiqueta </div> (se esperaba </h>)
15 - Cierre inesperado o incorrecto de etiqueta </div> (se esperaba </h>)
16 - Cierre inesperado o incorrecto de etiqueta </nav> (se esperaba </h>)
17 - Cierre inesperado o incorrecto de etiqueta </main> (se esperaba </h>)
18 - Cierre inesperado o incorrecto de etiqueta </body> (se esperaba </h>)
19 - Cierre inesperado o incorrecto de etiqueta </html> (se esperaba </h>)
20
21 - Etiqueta abierta sin cerrar: <h>
22 - Etiqueta abierta sin cerrar: <h>
23 - Etiqueta abierta sin cerrar: <h>
24 - Etiqueta abierta sin cerrar: <div>
25 - Etiqueta abierta sin cerrar: <div>
26 - Etiqueta abierta sin cerrar: <nav>
27 - Etiqueta abierta sin cerrar: <h>
28 - Etiqueta abierta sin cerrar: <div>
29 - Etiqueta abierta sin cerrar: <main>
30 - Etiqueta abierta sin cerrar: <body>
31 - Etiqueta abierta sin cerrar: <fefuncb>
32 - Etiqueta abierta sin cerrar: <fefuncb>
33 - Etiqueta abierta sin cerrar: <fefuncb>
34 - Etiqueta abierta sin cerrar: <fecomponenttransfer>
35 - Etiqueta abierta sin cerrar: <filter>
36 - Etiqueta abierta sin cerrar: <svg>
37 - Etiqueta abierta sin cerrar: <fefuncb>
38 - Etiqueta abierta sin cerrar: <fefuncb>
39 - Etiqueta abierta sin cerrar: <fefuncb>
40 - Etiqueta abierta sin cerrar: <fefuncb>
41 - Etiqueta abierta sin cerrar: <fecomponenttransfer>
42 - Etiqueta abierta sin cerrar: <filter>
43 - Etiqueta abierta sin cerrar: <svg>
44 - Etiqueta abierta sin cerrar: <path>
45 - Etiqueta abierta sin cerrar: <svg>
46 - Etiqueta abierta sin cerrar: <style>
47 - Etiqueta abierta sin cerrar: <head>
48 - Etiqueta abierta sin cerrar: <html>
49
50 [✓] Análisis completado. URLs guardadas en 'urls_extraidas.txt'.
51 PS C:\Users\USUARIO\Documents\GitHub\Practica_teoría_computacion>
```

```
1 import requests
2 from bs4 import BeautifulSoup
3 import os
4
5 # ...
6
7 PS C:\Users\USUARIO\Documents\GitHub\Practica_teoría_computacion> & C:\Users\USUARIO\AppData\Local\Programs\Python\Python311\python.exe c:\Users\USUARIO\Documents\GitHub\Practica_teoría_computacion/bs4_extractor.py
8
9 Analizando URL: https://www.example.com
10 Analizando URL: https://www.python.org
11 Analizando URL: https://www.wikipedia.org
12 Analizando archivo: prueba1.html
13 Analizando archivo: prueba2.html
14 Analizando archivo: prueba3.html
15 Analizando archivo: prueba4.html
16 Analizando archivo: prueba5.html
17 Analizando archivo: prueba6.html
18
19 [✓] example.com
20 Enlaces (1):
21 https://www.iana.org/domains/example
22
23 Imágenes (0):
24 Estadísticas de etiquetas:
25 <a>: 1
26 <img>: 0
27 <br>: 0
28 <div>: 1
29 <li>: 0
30 <ul>: 0
31 <p>: 2
32 <span>: 0
33 <table>: 0
34 <td>: 0
35 <tr>: 0
36
37 [✓] python.org
38 Enlaces (207):
39 #content
40 #python-network
41 /
42 https://www.python.org/psf/
43 https://docs.python.org
44 https://pypi.org/
45 /jobs/
46 /community/
47 #top
48 /
49 https://psfmember.org/civicrm/contribute/transact?reset=1&id=2
50 #site-map
51 #
52 javascript;;
53 javascript;;
54 javascript;;
55 #
```

```
bs4_extractor.py > ...
1 import requests
2 from bs4 import BeautifulSoup
3 import os

https://status.python.org/
/bs4-landing/
/about/legal/
https://policies.python.org/python.org/Privacy-Notice/
Imágenes (1):
/static/img/python-logo.png
Estadísticas de etiquetas:
<a>: 207
<img>: 1
<br>: 0
<div>: 48
<li>: 162
<ul>: 28
<p>: 23
<span>: 70
<table>: 1
<td>: 1
<tr>: 1

Wikipedia.org
Enlaces (370):
//en.wikipedia.org/
//ja.wikipedia.org/
//ru.wikipedia.org/
//de.wikipedia.org/
//es.wikipedia.org/
//fr.wikipedia.org/
//zh.wikipedia.org/
//it.wikipedia.org/
//pt.wikipedia.org/
//pl.wikipedia.org/
//ar.wikipedia.org/
//de.wikipedia.org/
//en.wikipedia.org/
//es.wikipedia.org/
//fa.wikipedia.org/
//fr.wikipedia.org/
//it.wikipedia.org/
//ar2.wikipedia.org/
//nl.wikipedia.org/
//ja.wikipedia.org/
//pl.wikipedia.org/
//pt.wikipedia.org/
//ceb.wikipedia.org/
//sv.wikipedia.org/
//uk.wikipedia.org/
//vi.wikipedia.org/
```

```
bs4_extractor.py > ...
1 import requests
2 from bs4 import BeautifulSoup
3 import os

//www.wikisource.org/
//species.wikimedia.org/
//www.wikifunctions.org/
//meta.wikimedia.org/
https://creativecommons.org/licenses/by-sa/4.0/
https://foundation.wikimedia.org/wiki/Special:MyLanguage/Policy/Terms_of_Use
https://foundation.wikimedia.org/wiki/Special:MyLanguage/Policy/Privacy_policy
Imágenes (1):
portal/wikipedia.org/assets/img/wikipedia-logo-v2.png
Estadísticas de etiquetas:
<a>: 370
<img>: 1
<br>: 0
<div>: 87
<li>: 342
<ul>: 7
<p>: 2
<span>: 60
<table>: 0
<td>: 0
<tr>: 0

prueba1.html
Enlaces (0):
Imágenes (0):
Estadísticas de etiquetas:
<a>: 0
<img>: 0
<br>: 0
<div>: 0
<li>: 0
<ul>: 0
<p>: 0
<span>: 0
<table>: 0
<td>: 0
<tr>: 0

prueba2.html
Enlaces (1):
http://www.bbc.co.uk
Imágenes (1):
brushedsteel.jpg
Estadísticas de etiquetas:
<a>: 1
<img>: 1
<br>: 1
```

```
bs4_extractor.py > ...
1 import requests
2 from bs4 import BeautifulSoup
3 import os

<td>: 0
<tr>: 0

prueba2.html
Enlaces (1):
http://www.bbc.co.uk
Imágenes (1):
brushedsteel.jpg
Estadísticas de etiquetas:
<a>: 1
<img>: 1
<br>: 1
<div>: 0
<li>: 0
<ul>: 0
<p>: 1
<span>: 0
<table>: 0
<td>: 0
<tr>: 0

prueba3.html
Enlaces (2):
http://www.bbc.co.uk
http://www.ibm.com
Imágenes (1):
https://www.uspceu.com/Portals/_default/skins/uspceu-multi/assets/img/logos/logo-ceu-horizontal-positivo.png
Estadísticas de etiquetas:
<a>: 2
<img>: 1
<br>: 1
<div>: 0
<li>: 0
<ul>: 0
<p>: 1
<span>: 0
<table>: 0
<td>: 0
<tr>: 0

prueba4.html
Enlaces (1):
http://www.bbc.co.uk
Imágenes (1):
brushedsteel.jpg
Estadísticas de etiquetas:
<a>: 1
```

```
bs4_extractor.py > ...
1 import requests
2 from bs4 import BeautifulSoup
3 import os

<tr>: 0

prueba4.html
Enlaces (1):
http://www.bbc.co.uk
Imágenes (1):
brushedsteel.jpg
Estadísticas de etiquetas:
<a>: 1
<img>: 1
<br>: 1
<div>: 0
<li>: 0
<ul>: 0
<p>: 2
<span>: 0
<table>: 0
<td>: 0
<tr>: 0

prueba5.html
Enlaces (1):
http://www.bbc.co.uk
Imágenes (1):
brushedsteel.jpg
Estadísticas de etiquetas:
<a>: 1
<img>: 1
<br>: 1
<div>: 0
<li>: 0
<ul>: 0
<p>: 1
<span>: 0
<table>: 0
<td>: 0
<tr>: 0

prueba6.html
Enlaces (48):
/refnav_logo
/customerpreferences/editis=UTF8preferencesReturnUrl=2Pref_topnav_lang
/customerpreferences/editis=UTF8preferencesReturnUrl=2Pref_topnav_lang
/gp/css/orderhistoryref_nnav_orders_first
https://www.amazon.es/gp/cart/view.htmlref_nnav_cart
/gp/sitedirectoryref_nnav_em_js_disabled
/hz/contactus/accessibility
```